

Rapport de TP : Test d'intrusion – Phase I :

Reconnaissance

Auteur : Raymond CHELLE

Formation : BTS SIO (SISR)

Date : 22/09/2026

Sommaire :

Rapport de TP : Test d'intrusion – Phase I : Reconnaissance	1
I. Introduction	1
II. Le travail à réaliser	2
Exercice 1 : Configuration et connectivité réseau	2
1. Identification des configurations IP	2
2. Test de connectivité (Ping) et pare-feu	3
Exercice 2 : Documentation et organisation sous CherryTree	4
Exercice 5 : Déploiement et reconnaissance avec SpiderFoot	6
1. Configuration du proxy et démarrage	6
2. Scan de la machine cible Windows 7 (scan_victime_seven3)	7
Exercice 6 : Bug Bounty (HackerOne) et audit d'un serveur Web local	9
1. Rôle et activité de HackerOne	9
2. Démarrage et scan du serveur Web local (XAMPP / Apache)	9
III. Conclusion	10

I. Introduction

Dans le cadre de ce TP sur les tests d'intrusion, l'objectif est de réaliser la première étape d'un audit : **la phase de reconnaissance**. Cette étape consiste à récolter un maximum d'informations sur la cible, de

manière passive (sans la solliciter directement, via des sources ouvertes / OSINT) ou active (en envoyant des requêtes et scans sur le réseau).

Nous mettons également en place une méthodologie de documentation structurée avec l'outil **CherryTree** et testons un outil d'OSINT et de scan automatisé (**SpiderFoot**).

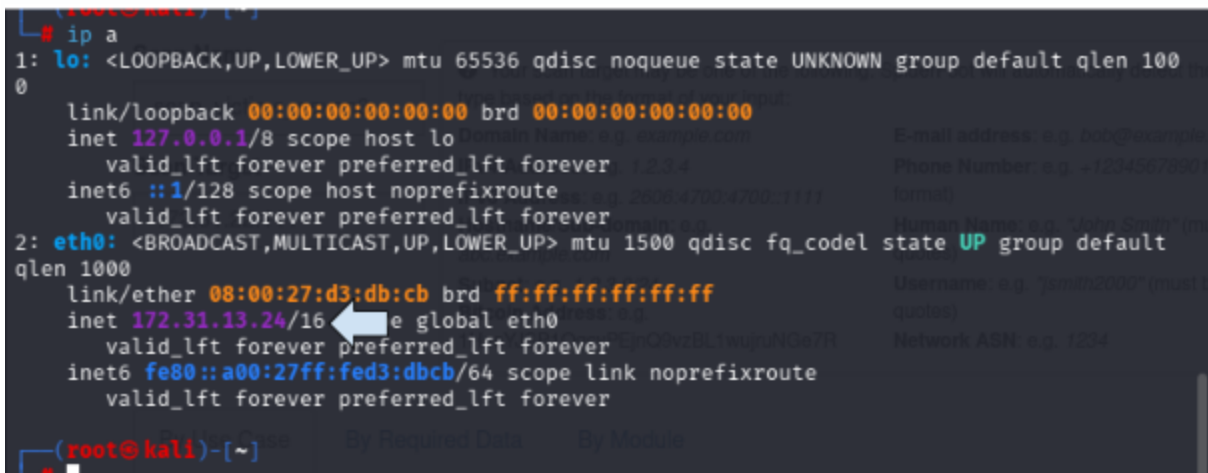
II. Le travail à réaliser

Exercice 1 : Configuration et connectivité réseau

1. Identification des configurations IP

Pour commencer, nous relevons les adresses IP et paramètres réseaux des deux machines virtuelles :

- **Machine attaquante (Kali Linux)** : IP 172.31.13.24/16 sur l'interface eth0 (Adresse MAC : 08:00:27:03:db:cb).
- **Machine cible (Windows 7)** : IP 172.31.22.111/16 (Masque : 255.255.0.0, Passerelle : 172.31.255.254).



```
(root@kali) [~]
# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host localhost
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d3:db:cb brd ff:ff:ff:ff:ff:ff
    inet 172.31.13.24/16 e global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fed3:dbcb/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(root@kali) [~]
```

E-mail address: e.g. bob@example.com
Phone Number: e.g. +1234567890
Email: e.g. bob@example.com
Human Name: e.g. "John Doe" (must be quoted)
Username: e.g. "john2000" (must be quoted)
Network ASN: e.g. 1234

```

Carte Ethernet Connexion au réseau local :
  Suffixe DNS propre à la connexion. . . . :
  Adresse IPv6 de liaison locale. . . . . : fe80::79ef:f12d:f05:67a3%11
  Adresse IPv4. . . . . : 172.31.22.111
  Masque de sous-réseau. . . . . : 255.255.0.0
  Passerelle par défaut. . . . . : 172.31.255.254

Carte Tunnel isatap.<23ECA039-217A-48ED-BB4B-9E7266F60B87> :
  Statut du média. . . . . : Média déconnecté
  Suffixe DNS propre à la connexion. . . . :
C:\Users\ubayuan\

```

2. Test de connectivité (Ping) et pare-feu

Lors d'un premier test de connectivité via la commande `ping -c 4 172.31.22.111` depuis la machine Kali Linux, aucun paquet n'est reçu (100% de perte).

Ce comportement s'explique par la configuration par défaut du **pare-feu Windows**, qui bloque les requêtes ICMP Echo Request pour empêcher la découverte de la machine lors d'un balayage réseau.

```

(root@kali)-[~]
# ping -c 4 172.31.22.111
PING 172.31.22.111 (172.31.22.111) 56(84) bytes of data.

— 172.31.22.111 ping statistics —
4 packets transmitted, 0 received, 100% packet loss, time 3069ms

```

Après désactivation du pare-feu sur Windows 7, la commande `ping -c 4 172.31.22.111` est relancée : les 4 paquets sont transmis et reçus sans aucune perte, avec un temps moyen de réponse de 0.34 ms. La liaison réseau est ainsi validée.

```
(root@kali)-[~]
# ping -c 4 172.31.22.111
PING 172.31.22.111 (172.31.22.111) 56(84) bytes of data.
64 bytes from 172.31.22.111: icmp_seq=1 ttl=128 time=0.588 ms
64 bytes from 172.31.22.111: icmp_seq=2 ttl=128 time=0.233 ms
64 bytes from 172.31.22.111: icmp_seq=3 ttl=128 time=0.275 ms
64 bytes from 172.31.22.111: icmp_seq=4 ttl=128 time=0.278 ms

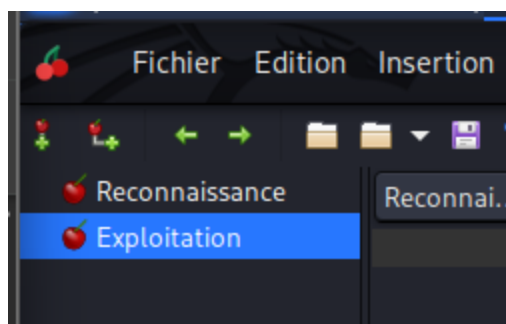
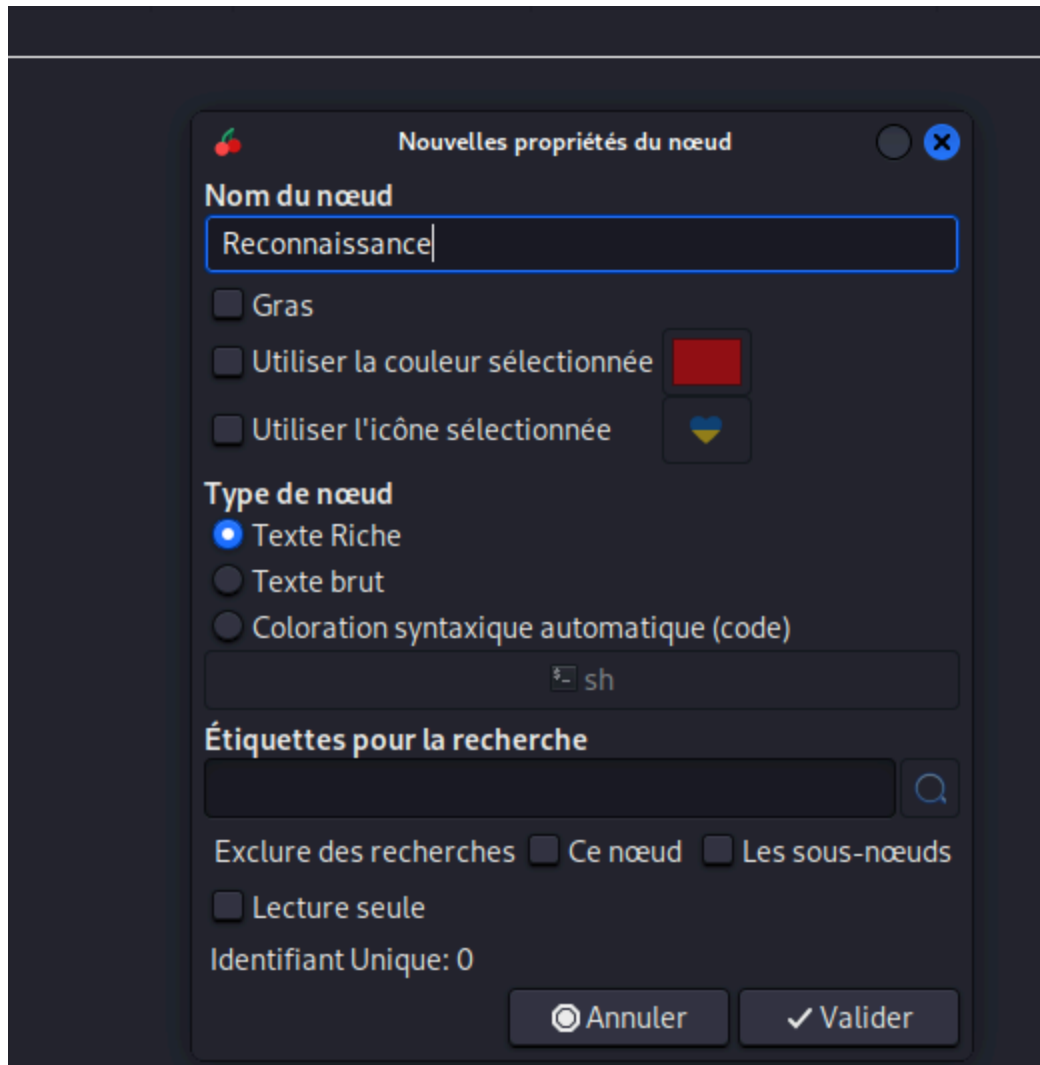
— 172.31.22.111 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3069ms
rtt min/avg/max/mdev = 0.233/0.343/0.588/0.142 ms
```

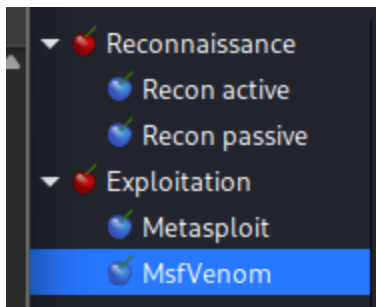
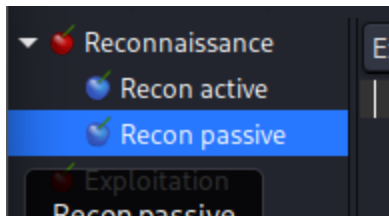
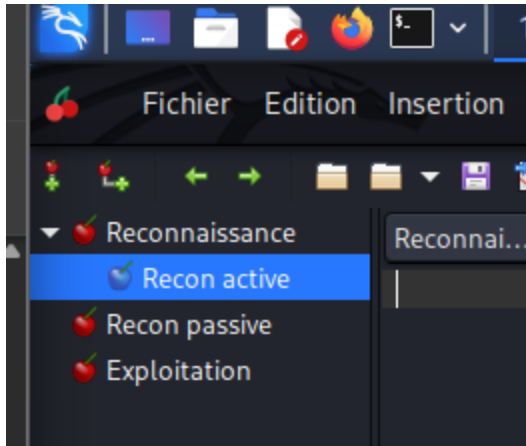
Exercice 2 : Documentation et organisation sous CherryTree

La documentation pas à pas étant essentielle lors d'un test d'intrusion afin de consigner les commandes, outils utilisés et résultats pour le rapport final, nous utilisons **CherryTree** préinstallé sur Kali Linux.

Nous avons structuré notre bloc-notes avec les nœuds et sous-nœuds suivants en format texte riche :

- **Reconnaissance** (nœud principal)
 - **Recon active** (pour consigner les scans réseau directs : Nmap, SpiderFoot, etc.)
 - **Recon passive** (pour la collecte d'informations en sources ouvertes, réseaux sociaux, bases de fuites)
- **Exploitation** (nœud principal)
 - **Metasploit** (préparation et configuration des exploits)
 - **MsfVenom** (génération de charges utiles / payloads)





Exercice 5 : Déploiement et reconnaissance avec SpiderFoot

1. Configuration du proxy et démarrage

Avant d'exécuter SpiderFoot, nous configurons les variables d'environnement du proxy dans le terminal Kali :

```
export http_proxy="http://172.16.63.130:3128"  
export https_proxy="http://172.16.63.130:3128"
```

Puis nous démarrons SpiderFoot avec la commande :

```
spiderfoot -l 172.31.13.24:5001
```

L'option -l (pour *listen*) permet de spécifier l'adresse IP et le port d'écoute sur lesquels le serveur web

local de l'application est démarré, rendant l'interface graphique accessible depuis un navigateur web à l'adresse 172.31.13.24:5001.

```
(root@kali)-[~]
# export http_proxy="http://172.16.63.130:3128"

(root@kali)-[~]
# export https_proxy="http://172.16.63.130:3128"

(root@kali)-[~]
# spiderfoot -l 172.31.13.24:5001

Your scan target may be one of the following. SpiderFoot will automatically detect the
type based on the format of your input.
2026-09-22 11:08:02,833 [INFO] sf : Starting web server at 172.31.13.24:5001 ...
2026-09-22 11:08:02,850 [WARNING] sf : In Name: e.g. example.com E-mail address: e.g. bob@example.com
***** Phone Number: e.g. +12345678901 *****
Warning: passwd file contains no passwords. Authentication disabled.
Please consider adding authentication to protect this instance!
Refer to https://www.spiderfoot.net/documentation/#security.
***** Username: e.g. "jsmith2000" (must be in quotes) *****
***** Subnet: e.g. 192.168.0/24 *****
***** Bitcoin Address: e.g. 1A1zP1eP5QGefi2DMPTfKY5 *****
***** Network ASN: e.g. 1234 *****

Use SpiderFoot by starting your web browser of choice and
browse to http://172.31.13.24:5001/
*****
```

2. Scan de la machine cible Windows 7 (scan_victim_seven3)

Un scan complet de la cible 172.31.22.111 est initié. Le module sf_portscan_tcp détecte deux ports ouverts :

- **Port 139 / TCP** (NetBIOS Session Service)
- **Port 445 / TCP** (Microsoft-DS / SMB)

La présence du port 445 confirme un service de partage Windows actif, constituant une cible d'analyse privilégiée pour la recherche de vulnérabilités système.

☐	Data Element	Source Data Element	Source Module	Identified
☐	172.31.22.111:130	172.31.22.111	sfp_portscan_tcp	2020-09-22 11:10:10
☐	172.31.22.111:445	172.31.22.111	sfp_portscan_tcp	2020-09-22 11:10:26

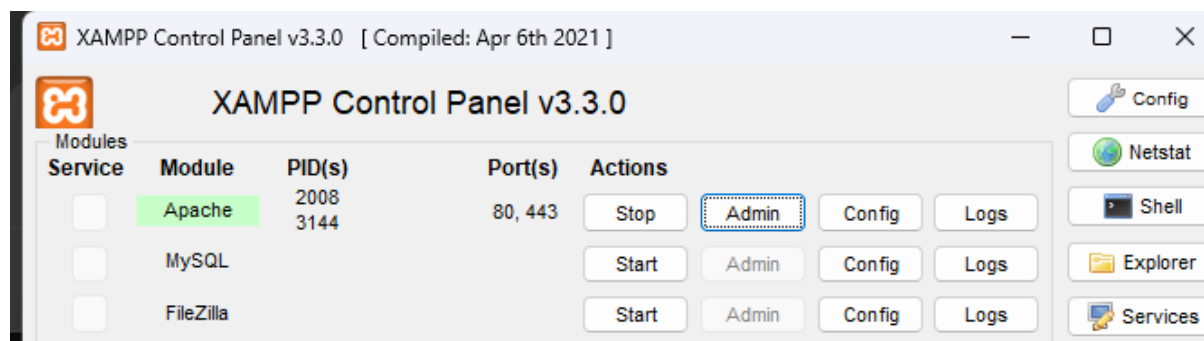
Exercice 6 : Bug Bounty (HackerOne) et audit d'un serveur Web local

1. Rôle et activité de HackerOne

HackerOne est une plateforme intermédiaire de Bug Bounty et de divulgation coordonnée de vulnérabilités (VDP). Elle met en relation des organisations avec une communauté mondiale de hackers éthiques (white hats). Les entreprises y définissent le périmètre précis de leurs actifs numériques à tester ; les chercheurs en sécurité traquent les failles, les signalent et perçoivent en retour des récompenses financières (primes) ainsi que des points de réputation.

2. Démarrage et scan du serveur Web local (XAMPP / Apache)

Sur la machine cible (**Windows 7**), le service Apache est démarré via le panneau de contrôle XAMPP, occupant les ports standard 80 et 443.



Nous lançons ensuite un scan SpiderFoot (finalscan) ciblant l'IP 172.31.13.1. Le scan identifie l'ouverture des ports :

- **Port 80 / TCP** (HTTP non sécurisé)
- **Port 443 / TCP** (HTTPS sécurisé avec chiffrement SSL/TLS)

← → ↻ 🏠 Not Secure http://172.31.22.109/newscan 80% ☆ 👤 📁 ☰

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB >>

spiderfoot New Scan Scans Settings Light Mode About

New Scan

Scan Name
finalscan

Scan Target
172.31.13.1

📌 Your scan target may be one of the following. SpiderFoot will automatically detect the target type based on the format of your input:

Domain Name: e.g. example.com	E-mail address: e.g. bob@example.com
IPv4 Address: e.g. 1.2.3.4	Phone Number: e.g. +12345678901 (E.164 format)
IPv6 Address: e.g. 2606:4700:4700::1111	Human Name: e.g. "John Smith" (must be in quotes)
Hostname/Sub-domain: e.g. abc.example.com	Username: e.g. "jsmith2000" (must be in quotes)
Subnet: e.g. 1.2.3.0/24	Network ASN: e.g. 1234
Bitcoin Address: e.g. 1HesYJSP1QqcyPEjnQ9vzBL1wujruNGe7R	

By Use Case By Required Data By Module

final FINISHED

Summary Correlations Browse Graph Scan Settings Log

Type	Unique Data Elements	Total Data Elements	Last Data Element
IP Address	1	1	2026-09-29 08:30:31
Open TCP Port	2	2	2026-09-29 08:31:53
Raw Data from RIRs/APIs	1	1	2026-09-29 08:30:37

final FINISHED

Summary Correlations Browse Graph Scan Settings Log

Browse / Open TCP Port

	Data Element	Source Data Element	Source Module	Identified
☐	172.31.13.1:443	172.31.13.1	sfp_portscan_tcp	2026-09-29 08:30:52
☐	172.31.13.1:80	172.31.13.1	sfp_portscan_tcp	2026-09-29 08:31:53

III. Conclusion

Ce TP a permis d'aborder concrètement la phase de reconnaissance d'un test d'intrusion. Nous avons vérifié la connectivité et géré les restrictions de pare-feu, automatisé la cartographie des ports et services grâce à SpiderFoot (SMB 139/445, Web 80/443), et compris l'importance de structurer rigoureusement ses notes sous CherryTree en vue de la restitution finale d'un audit de sécurité.